Verified Legal Reasoning: From Natural Language to Formal Proof

Eduardo Aguilar Pelaez[1,2]

[1] Legal Engine Ltd, London, UK
edu@legalengine.co.uk
[2] Imperial College London, London, UK
e.aguilar@imperial.ac.uk

Abstract. Legal AI systems increasingly serve as transducers – converting unstructured statutory text into structured representations – but they lack a verification layer capable of distinguishing what the type checker can certify from what requires human judgement. This paper presents LegalLean, a framework for verified legal reasoning built on Lean 4 dependent types. The central contribution is the `FormalisationBoundary` type and the `RequiresHumanDetermination` typeclass, which together encode Hart's "open texture" distinction as a first-class object in the type system rather than an external limitation on formalisation. We demonstrate the framework across three case studies: IRC §163(h) mortgage interest deduction, the O-1A extraordinary ability visa (8 CFR §204.5(h)), and the Australian Telecommunications Consumer Protections Code (C628:2012). The system produces 44 machine-verified theorems with zero unresolved `sorry` placeholders. The headline result is `monotonicity_break_general`: a machine-checked proof that the Tax Cuts and Jobs Act 2017 removed a previously valid deduction permission for all home equity interest payments – a formally verified instance of non-monotonic statutory change. We compare LegalLean against four prior systems (DDL, Catala, L4L, LegalRuleML) and discuss honest limitations.

Keywords: Dependent types · Legal formalisation · Lean 4 · Open texture · Formal verification · Defeasible reasoning

1 Introduction

Google DeepMind's description of its language model as a "transducer and verifier" articulates a productive division of labour: the language model converts messy natural-language input into structured form, while a separate verification layer checks the logic. This framing, applied to law, identifies exactly what is missing from current legal AI systems. Systems that extract rules from statutes, answer legal queries, or assess compliance are, in this terminology, transducers. They produce structured representations. None of them includes a verifier in the formal sense: a component that provides machine-checkable guarantees about the reasoning it performs.

The gap matters because legal reasoning has a well-understood failure mode. Hart's analysis in *The Concept of Law* [5] identified that legal concepts have a core of settled meaning and a penumbra of uncertainty. A formalisation that does not distinguish these zones does not merely lack nuance – it silently misrepresents where its own confidence ends. When a compliance system returns "non-compliant", the user cannot tell whether that verdict rests on a formally verifiable ground (the payment was not made by an individual) or a contestable judgement (the property may not qualify as a "qualified residence"). Both appear as outputs; only one is machine-checkable.

Existing approaches address parts of this problem but not the whole. Catala [10] is a functional programming language designed for statutes: it computes legal outcomes deterministically but has no mechanism for representing open texture – the code either handles a case or raises a runtime error. Governatori's Defeasible Deontic Logic (DDL) [3, 4] encodes legal rules as defeasible clauses with priority relations, handling non-monotonicity well but providing no machine-checkable proofs and treating vague predicates as ordinary literals. L4L (Logic for Law) [6] uses dependent types in a style close to ours and produces SMT-backed proofs, but the SMT layer is not fully transparent: the proof obligations are discharged by a solver whose output is not an auditable term. LegalRuleML [12] is a rich XML vocabulary for regulatory knowledge but provides no automated reasoning whatsoever. None of these systems makes the formalisation boundary itself a typed, auditable object.

LegalLean takes a different approach. It uses Lean 4 [11] dependent types, where the Curry-Howard correspondence [2, 7] establishes that propositions are types and proofs are terms. A legal conclusion – "this interest payment is not deductible" – is a type. A proof of that conclusion is a term of that type. If the term type-checks, the conclusion is verified; if it does not, the error is explicit and localised. Crucially, when a legal condition requires human determination (because the underlying predicate is vague or discretionary), LegalLean does not attempt to discharge it. Instead, it wraps the proposition in a `FormalisationBoundary` value that carries machine-readable metadata about which institution must determine it and why.

An important architectural distinction: LegalLean is a verification layer, not an inference engine. The case study functions (such as `isDeductible` for IRC §163(h)) are formalisations of statutory logic written by a human or language model. They are programs to be verified, not derivations from a rule base. The value of Lean 4 is not that it replaces the formaliser but that it catches errors in the formalisation: if the encoded logic is inconsistent with the declared types, the type checker rejects it. A generic defeasible reasoning solver (in the style of DDL's proof theory) would be a valuable complement but serves a different role. DDL computes which conclusions follow from a set of rules; LegalLean verifies that a specific encoding of those rules is internally consistent and correctly typed. The two are complementary, not competitive.

The contributions of this paper are:

1. The `FormalisationBoundary` inductive type and the `RequiresHumanDetermination` typeclass (with `Vague` and `Discretionary` subclasses), providing the first formal treatment of Hart's open texture as a machine-checkable type-level distinction.
2. A verified encoding of IRC §163(h) including `monotonicity_break_general`, a machine-checked proof that the TCJA 2017 constitutes a non-monotonic statutory change: for every home equity payment satisfying the pre-TCJA conditions, the legal outcome transitions from `boundary` (human determination required) to `formal False` (machine-verified rejection).
3. A dependent-type encoding of the O-1A visa "3 of 8" threshold using `AllDistinct` lists with length constraints, together with a `mixed` formalisation of the Kazarian two-part test.
4. A structured comparison of LegalLean against DDL [3], Catala [10], L4L [6], and LegalRuleML [12] across seven dimensions.

All source code is available and compiles against Lean 4 v4.24.0 (the Aristotle release) with no Mathlib dependency.

2 Background

2.1 Hart's Open Texture

In *The Concept of Law* (1961), H.L.A. Hart [5] observed that legal concepts have a "core of settled meaning" and a "penumbra of uncertainty". The canonical example is a rule prohibiting vehicles in a park: a car is unambiguously within the rule's core; a toy pedal car is in the penumbra. The penumbral question cannot be resolved by logic alone – it requires a judgement about the rule's purpose and the case's particulars. Hart called this irreducible property "open texture": the meaning of legal terms cannot be fully determined in advance, and no formalisation can eliminate the need for human judgement at the boundary.

Prior legal knowledge representation systems treat open texture as a limitation imposed from outside: the formalisation goes as far as it can, and the rest is left to the practitioner. LegalLean treats it as a property to be represented inside the system.

2.2 Default Logic and Defeasibility

Legal reasoning is non-monotonic: adding new rules or evidence can retract previously valid conclusions. Lawsky's "A Logic for Statutes" [8] provides a careful analysis of how Reiter's default logic [13] applies to statutory interpretation, demonstrating that statutes encode default rules of the form: "if conditions hold and no exception applies, the conclusion follows." The exceptions are themselves defeasible, creating chains of defeat.

LegalLean represents defeasibility through the `Defeats` structure and `DefeasiblyHolds` proposition type. Priority ordering (*lex posterior*, *lex specialis*) is encoded in the `Priority` field of each `LegalRule`. The key departure from prior work is that

defeat relations are typed: the `Defeats` structure carries a `reason` field distinguishing *lex posterior* from *lex specialis*, and its `strict` field distinguishes hard from soft defeat.

2.3 Curry-Howard and Lean 4

The Curry-Howard correspondence [2, 7] establishes an isomorphism between propositions and types: a proposition `P` is inhabited (has a term of that type) if and only if it is provable. In Lean 4 [11], this means that constructing a value of type `EvidenceFor P` is literally the same operation as proving `P`. Lean 4's kernel verifier checks all proof terms, making the guarantee machine-checkable in a stronger sense than SMT: the checker is a small, auditable kernel, not an opaque solver.

Lean 4's dependent type system allows the type of a term to depend on its value. This is essential for legal formalisation: the type of evidence required for "meets 3 of 8 criteria" depends on which specific criteria are claimed, and `MeetsThreeCriteria` encodes this directly via an `AllDistinct` list of `Criterion` values with a `cardinal` proof that `satisfied.length >= 3`.

3 The LegalLean Framework

3.1 Core Types

The framework begins with three foundational types defined in `LegalLean.Core`:

```
inductive Deontic where
  | permitted    : Deontic
  | prohibited   : Deontic
  | obligatory   : Deontic
  deriving Repr, BEq, Inhabited, DecidableEq
```

The deontic modalities cover the three fundamental normative positions. Lean 4's `DecidableEq` derivation means that equality of modalities is decidable, enabling automated conflict detection in theorems (for instance, `R1_BaseDeduction.modality \ne R2_TCJAPr` is proved by `simp`).

```
structure LegalRule (α : Type) where
  id          : String
  description : String
  modality    : Deontic
  priority    : Priority
  conclusion  : α → Prop
  defeatedBy  : List String
```

`LegalRule` is parameterised by the type α of the entity it governs: `LegalRule InterestPayment` for IRC §163(h) rules, `LegalRule Provider` for TCP Code rules. The `conclusion` field is a function from α to `Prop`, encoding the rule's conclusion as a dependent proposition. This is more expressive than DDL's flat first-order signature.

```
structure EvidenceFor (P : Prop) where
  source   : String
  artifact : Option String
  witness  : P
```

`EvidenceFor P` is the propositions-as-types materialisation of legal evidence: the `witness` field literally contains the proof of `P`. Constructing an `EvidenceFor P` value is constructing a proof. The `source` and `artifact` fields provide provenance that survives code evolution – they are not erased at compile time.

3.2 The FormalisationBoundary Type

`FormalisationBoundary` is the novel contribution of this framework:

```
inductive FormalisationBoundary (P : Sort _) where
  | formal   (proof : P)                                  :
      FormalisationBoundary P
  | boundary (reason : String) (authority : String)       :
      FormalisationBoundary P
  | mixed    (formalPart : String) (humanPart : String)   :
      FormalisationBoundary P
```

The three constructors correspond to three epistemic positions about a legal proposition `P`:

- `formal proof`: the type checker alone can verify `P`. No human determination is required. The `proof` term is a machine-checked certificate.
- `boundary reason authority`: `P` requires human determination. The `reason` string names the specific open-texture condition; `authority` names the institution responsible.
- `mixed formalPart humanPart`: `P` is partially formal and partially boundary. Part 1 of the Kazarian two-part test is `formal` (Lean verified the criteria count); Part 2 is `boundary` (USCIS holistic determination).

3.3 The RequiresHumanDetermination Typeclass

The `FormalisationBoundary` type marks where the boundary falls. The `RequiresHumanDetermination` typeclass marks which propositions are intrinsically at the boundary:

```
class RequiresHumanDetermination (P : Prop) where
  reason    : String
  authority : String

class Vague (P : Prop) extends RequiresHumanDetermination P
    where
  core      : String
  penumbra  : String
```

```
class Discretionary (P : Prop) extends
    RequiresHumanDetermination P where
  scope       : String
  constraints : List String
```

`Vague` captures Hart's core/penumbra distinction directly. `core` describes settled cases; `penumbra` describes the contested region. `Discretionary` captures a different phenomenon: not conceptual vagueness but institutional authority. A USCIS adjudicator's determination of "extraordinary ability" is not vague in Hart's sense – the adjudicator has the power to decide. The typeclass distinguishes these two sources of non-formalisability.

Both typeclasses constrain proof search: any proposition with a `Vague` or `Discretionary` instance cannot be discharged by the type checker without an explicit axiom. In the case studies, open-texture propositions such as `HasExtraordinaryAbility` and `IsQualifiedResidence` are introduced as axioms – honest markers that the type checker cannot verify them. The typeclass instances then carry the auditable metadata explaining why.

The use of axioms for open-texture propositions requires justification. An axiom in Lean 4 is a proposition asserted without proof; it is not verified by the kernel. One might object that this introduces a soundness gap: if the axioms are inconsistent, any theorem built on them is vacuously true. This objection is correct in principle but misdirected in practice. The axioms are not intended to be discharged; they represent propositions that *should not* be machine-verifiable. A system that could prove `HasExtraordinaryAbility a` for a given applicant `a` would be claiming to have automated a determination that the US Congress assigned to USCIS adjudicators. The axiom marks the boundary between what the type checker should certify and what it should defer. The theorems built on these axioms do not depend on their truth – they depend on their *structure*. The monotonicity break theorem, for instance, does not use any open-texture axiom; it reasons entirely about the formalisable conditions. This is the key architectural difference from all prior systems: the formalisation boundary is not a gap in the encoding but a typed, first-class object.

4 Case Studies

4.1 IRC §163(h): Mortgage Interest Deduction

IRC §163(h) (26 U.S.C. §163(h)) is the provision of the US Internal Revenue Code governing the deductibility of qualified residence interest. It was chosen because Lawsky [8] used it as the gold standard case study for default logic applied to statutes. The rule structure consists of three rules, four defeat relations forming a three-level chain, and three open-texture terms.

The three rules in `LegalLean.CaseStudies.IRC163h` are:

- **R1** (`R1_BaseDeduction`): Taxpayers are permitted to deduct interest on acquisition indebtedness secured by a qualified residence. Priority level 1 ("statute").

- **R2** (`R2_TCJAProhibition`): Home equity interest deduction is prohibited for taxable years beginning after 15 December 2017 (the TCJA). Priority level 2 ("statute (TCJA 2017, *lex posterior*)").
- **R3** (`R3_GrandfatherRule`): Acquisition indebtedness outstanding on 13 October 1987 is grandfathered from current dollar limits. Priority level 3 ("statute (*lex specialis*, grandfather clause)").

The four defeat relations are: R2 defeats R1 (*lex posterior*); E1.1 (dollar limit) defeats R1; E1.2 (home equity dollar limit) defeats R1; and R3 defeats E1.1. The third of these creates a three-level defeasibility chain: R3 defeats E1.1, which defeats R1 – defeat of the limit restores the base permission. This chain cannot be represented by a flat priority ordering; it requires the full defeasibility machinery.

Open-texture terms are encoded as axioms with `Vague` instances. `IsQualifiedResidence` is declared as an axiom, with the instance documenting the core (principal residence plus one other property used as a residence) and the penumbra (mobile homes, timeshares, boats with habitability facilities, fractional ownership). `IsSubstantialImprovement` receives the same treatment. `MixedUseAllocation` receives a `Discretionary` instance referencing IRS tracing rules under Treas. Reg. §1.163-8T.

The compliance function `isDeductible` returns a `FormalisationBoundary Prop`. For unlicensed entities, post-TCJA home equity, and unsecured debt, it returns `formal False` – machine-verified rejection. For acquisition indebtedness within limits, it returns `boundary` with the authority "IRS / Tax Court".

The Monotonicity Break Theorem. The headline result of this case study is `monotonicity_break_general`:

```
theorem monotonicity_break_general
    (payment : InterestPayment) (status : FilingStatus)
    (h_individual : payment.paidByIndividual = true)
    (h_secured    : payment.indebtedness.securedByResidence
        = true)
    (h_purpose    : payment.indebtedness.purpose =
       IndebtednessPurpose.other)
    (h_within     : withinHomeEquityLimit payment.
       indebtedness status = true)
    : (∃ r a, isDeductible payment ⟨2017⟩ status =
                FormalisationBoundary.boundary r a) ∧
      (isDeductible payment ⟨2018⟩ status =
                FormalisationBoundary.formal False) := by
  constructor
  · simp [isDeductible, h_individual, h_secured, h_purpose,
      h_within]
  · simp [isDeductible, h_individual, h_secured, h_purpose]
```

This theorem states, for all home equity interest payments by individual taxpayers on secured property within the applicable dollar limits: the pre-TCJA out-

come (year = 2017) is `boundary` (potentially deductible; human determination of "qualified residence" required), while the post-TCJA outcome (year = 2018) is `formal False` (automatically rejected; machine-verified). The TCJA added a rule to the legal system that removed a previously available permission. Lean 4 verifies this as a structural property of the encoding, not merely a test case.

A companion corollary `pre_tcja_not_auto_rejected` confirms the strictness of the break: the pre-TCJA outcome is provably not `formal False`, making the transition genuinely from "possibly permitted" to "certainly prohibited."

It is worth being explicit about what this theorem proves and does not prove. It proves a structural property of the formalisation: that the `isDeductible` function, as encoded, produces these two outcomes for these two inputs. It does not prove that the formalisation is a complete representation of IRC §163(h) – such a claim would require independent legal audit. The theorem's value is that it is machine-checked: the outcome cannot diverge silently from the encoding.

The proof's brevity (two `simp` calls) does not indicate a trivially encoded result. The `simp` tactic unfolds `isDeductible`, substitutes the hypotheses, and evaluates the `if` branches. The non-triviality is in the *universality*: the theorem quantifies over all `InterestPayment` values and all `FilingStatus` values satisfying the four preconditions. A test case checks one input; this theorem checks the entire input space. The preconditions are exactly the conditions under which the monotonicity break is legally relevant (individual taxpayer, secured home equity debt within limits), and the universally quantified conclusion is that the legal outcome changes at the 2018 boundary for every such payment. This structural coverage is what distinguishes a verified theorem from a unit test, even when both share the same tactic.

4.2 O-1A Visa: "3 of 8" as Dependent Types

The O-1A extraordinary ability visa (8 CFR §204.5(h)) requires demonstrating "extraordinary ability" by meeting at least three of eight evidentiary criteria. This is a natural dependent-type problem: the "3 of 8" threshold is a cardinality constraint on a set of satisfied criteria, and Lean 4 can encode this constraint at the type level.

The eight criteria are defined as an inductive type `Criterion` with `DecidableEq`, allowing duplicate-free lists. Since the case study avoids Mathlib, cardinality is encoded without `Finset`:

```
inductive AllDistinct {α : Type} [DecidableEq α] : List α →
    Prop
  | nil  : AllDistinct []
  | cons : ∀ {x : α} {xs : List α}, x ∉ xs → AllDistinct xs
              → AllDistinct (x :: xs)

structure MeetsThreeCriteria (a : Applicant) where
  satisfied : List Criterion
  distinct  : AllDistinct satisfied
  cardinal  : satisfied.length ≥ 3
```

```
  evidence  : ∀ c ∈ satisfied, LegalLean.EvidenceFor (
      criterionProp a c)
```

A value of type `MeetsThreeCriteria a` can only be constructed if one provides a duplicate-free list of at least three criteria together with `EvidenceFor` witnesses for each. The type checker enforces the threshold at compile time. This is precisely the propositions-as-types reading of "the applicant meets the threshold": inhabiting the type is proving the claim.

The Kazarian two-part test (*Kazarian v. USCIS*, 596 F.3d 1115, 9th Cir. 2010) is encoded by `determineEligibility`, which returns `FormalisationBoundary.mixed`: the `formalPart` records that Lean verified the criteria count; the `humanPart` names the USCIS adjudicator and the final merits determination as the non-formalisable component. The theorem `eligibility_always_mixed` proves that `determineEligibility` always returns `mixed` – never `formal` or `boundary` alone – which is a structural consequence of the Kazarian framework: Part 1 is always machine-checkable; Part 2 is always not.

4.3 TCP Code: Deontic Spectrum and DDL Comparison

The third case study encodes a kernel of the Australian Telecommunications Consumer Protections Code (C628:2012), chosen for direct comparison with Governatori's DDL encoding of the same regulatory text [3, 4]. Three provisions are encoded: TCP 3.2 (contact details obligation), TCP 5.1 (Critical Information Summary obligation), and TCP 8.2 (excessive early termination fee prohibition). Each has a *lex specialis* defeat: small provider exemption, renewal exemption, and device repayment carve-out respectively.

The three provisions demonstrate the full deontic spectrum: TCP 3.2 imposes an obligation (providers must publish contact details); TCP 8.2 imposes a prohibition (providers must not charge excessive ETFs); and the defeat rules introduce permissions (small providers may use simplified requirements; device repayment fees are allowed). The theorems `contact_obligation_and_exemption_are_deontic_distinct` and `etf_prohibition_and_carveout_are_deontic_distinct` prove that the base rule and its defeating exception always have distinct modalities.

The `exemption_rules_outrank_base_rules` theorem verifies that defeating rules have strictly higher priority levels than the rules they defeat across all three provisions:

```
theorem exemption_rules_outrank_base_rules :
    R1_SmallProviderExemption.priority.level >
        R1_ContactObligation.priority.level ∧
    R2_RenewalExemption.priority.level >
        R2_CriticalInfoSummary.priority.level ∧
    R3_DeviceRepaymentCarveout.priority.level >
        R3_ExcessiveETFProhibited.priority.level
```

This is a structural soundness property: a defeat relation where the defeating rule had lower or equal priority would be incoherent.

The DDL encoding of the same provisions is documented as comments in the source file. For example, the contact obligation in DDL syntax:

```
d ⊗ (isLicensed(P)) ⇒ O(publishContactDetails(P))
r1 :- licensed(P), not small_provider(P).
r1_exc :- small_provider(P).
r1_exc > r1
```

The LegalLean encoding differs in four respects: (1) `FormalisationBoundary` distinguishes `prominently_published` from `is_licensed` as requiring human determination, rather than treating both as ordinary literals; (2) `LegalRule (α : Type)` parameterises rules by the entity type, producing a typed rather than flat signature; (3) `EvidenceFor P` requires a proof witness rather than mere assertion; (4) the `Vague` and `Discretionary` instances carry auditable metadata that is machine-readable, not just a comment.

5 Evaluation

5.1 Theorem Count and Properties

The system produces 44 machine-verified theorems with zero `sorry` placeholders across the three case studies. It is important to be honest about the nature of these theorems. Some are structural unit tests: `non_individual_never_deducts` is proved by `simp [isDeductible, h]` – a trivial path trace through a function. These theorems are valuable as regression tests but do not constitute substantive contributions in isolation.

Others are more substantive. `monotonicity_break_general` requires reasoning about two distinct legal regimes and proving a structural divergence. `defeat_chain_exists` proves that the defeat relation in IRC §163(h) has genuine depth – R3 defeats E1.1, which defeats R1, but R3 does not directly defeat R1 – capturing a legally meaningful subtlety about the grandfather clause. `no_mixed_outcomes` proves by full case analysis that IRC §163(h)'s `isDeductible` function never returns a `mixed` result, establishing a binary structure as a property of the statute rather than a design choice.

A faithfulness audit identified three areas where the encoding simplifies the actual law: (1) date comparisons use ISO 8601 string ordering rather than proper date arithmetic; (2) `criterionProp` now maps each O-1A criterion to its own distinct axiom (e.g. `HasExtraordinaryAbility`, `SalaryIsHigh`, `ContributionsAreMajor`), with each carrying a separate `Vague` or `Discretionary` instance that documents the specific core and penumbra for that criterion; (3) the IRC §163(h) encoding does not handle the full debt tracing rules of Treas. Reg. §1.163-8T, which require temporal tracking of how loan proceeds were used.

5.2 Comparison with Prior Systems

Table 1 compares LegalLean against four prior systems across seven dimensions on a 1–5 scale. As noted below, these scores should be treated with caution.

Table 1. Comparison of LegalLean with prior legal formalisation systems. Scores are on a 1–5 scale; higher is better. Scores reflect the dimensions on which LegalLean makes a genuine claim, not an objective ranking.

Dimension	**LegalLean**	**L4L**	**DDL**	**Catala**	**LegalRuleML**
Compositional semantics	5	3	4	4	2
Deontic modalities	5	4	5	2	4
Dependent types	5	5	1	1	1
Machine-checkable guarantees	5	3	3	4	1
Open texture encoding	5	3	4	2	2
Provenance tracking	5	5	4	3	4
Temporal reasoning	5	3	4	3	4
Total	**35**	**26**	**25**	**19**	**18**

These scores were generated as part of a structured comparison exercise and must be interpreted with appropriate scepticism. The scores for LegalLean are perfect (5/5 on every dimension), which reflects the dimensions being chosen partly to highlight LegalLean's strengths. A reviewer who weighted practical adoption, tool maturity, or non-expert usability would produce different rankings.

DDL [3, 4]: strong on deontic modalities and temporal reasoning; the most mature industrial-strength defeasible logic system for law. Its limitation is that it treats all conditions as first-order literals – `prominently_published(P)` is no different from `is_licensed(P)` in the DDL signature. There is no mechanism to mark the former as requiring institutional determination.

Catala [10]: designed specifically for tax and social benefit statutes, with excellent practical coverage. It computes legal outcomes rather than proving properties about them: Catala programs execute, they do not produce proof certificates. It has no representation of open texture. In LegalLean, `isDeductible` returns `FormalisationBoundary.boundary "qualified residence" "IRS / Tax Court"` – the system records that it cannot determine this condition and names the authority responsible. The distinction is not that LegalLean "handles" vagueness better, but that it represents the inability to handle it as a typed, auditable object rather than a silent gap.

L4L [6]: the closest prior work in spirit. L4L uses dependent types and produces proofs, but SMT-backed verification means the proof certificates are not auditable proof terms in the Curry-Howard sense. Its treatment of open texture relies on the SMT solver's inability to discharge certain goals rather than a typed annotation.

LegalRuleML [12]: a rich XML ontology with sophisticated modelling of deontic modalities and temporal applicability. It is a knowledge representation language, not a reasoning system – it provides no automated inference or proof production.

5.3 Limitations

Scalability: the three case studies cover nine provisions and fewer than 300 lines of Lean code each. Scaling to a complete statute would require substantially more proof engineering.

No LLM-to-Lean pipeline: the framework does not yet include an automated formalisation pipeline. The transducer half of the architecture – converting statutory text to Lean 4 – is currently manual.

Axiom dependence: the open-texture axioms are genuine axioms that cannot be proved by Lean's kernel. The framework is only as sound as the legal judgements behind the `Vague` and `Discretionary` instances.

No temporal algebra: the TCJA cutoff is encoded as a numeric comparison (`year.year >= 2018`), not via a proper temporal type with duration, retroactivity, and sunset provisions. A complete temporal logic layer remains future work.

Sorry-bypass gap: the type system does not prevent the construction `FormalisationBoundary.formal so` A careless user could mark a proposition as formally verified using Lean's `sorry` axiom. The VERIFICATION.md protocol addresses this by requiring `grep -r sorry` as part of the verification layer, but the type system itself does not enforce it. This is an engineering limitation, not a theoretical one.

Normative risk of formalisation: by encoding Hart's open texture as a typed object, this work makes the formalisation boundary tractable and auditable. This carries a normative risk that Lessig's "code is law" tradition identifies: making the boundary typed may create a false sense of completeness. A user who sees `FormalisationBoundary.boundary "requires IRS determination" "IRS / Tax Court"` might treat the annotation as sufficient engagement with the underlying vagueness, when in fact the core/penumbra analysis in the `Vague` instance is one interpretation among several. The system does not resolve open texture; it classifies and annotates it. Users must understand that the annotation is a map of the boundary, not a resolution of it. This limitation is intrinsic to any formalisation of vagueness and should be communicated clearly in any deployment context.

6 Related Work

Defeasible Deontic Logic (DDL) [3, 4]: the most complete prior work on defeasible reasoning in law. It provides a proof theory for defeasible rules with deontic modalities and has been applied to compliance checking in eHealth, business contracts, and regulatory codes. LegalLean's advance over DDL is not in defeasibility machinery but in (1) machine-checkable proof certificates, (2) the `FormalisationBoundary` typing of open texture, and (3) dependent types enabling evidence-as-inhabitants.

Catala [10]: the most practical prior system for statutory formalisation, with a growing library of French, US, and Polish tax law. Its design priority is executability: a Catala specification is a runnable programme that computes legal outcomes from facts. LegalLean's priority is verifiability. The two approaches are

complementary rather than competitive. A concrete example illustrates this complementarity: in IRC §163(h), the condition "property is a qualified residence" is vague in Hart's sense (the penumbra includes mobile homes, timeshares, and boats). In Catala, this condition must either be encoded as a Boolean input (the user asserts it) or omitted (the programme raises an error for unhandled cases). In LegalLean, `isDeductible` returns `FormalisationBoundary.boundary "qualified residence" "IRS / T` – the system records that it cannot determine this condition, names the authority responsible, and the `Vague` instance on `IsQualifiedResidence` documents the core and penumbra for audit. The distinction is not that LegalLean "handles" vagueness better, but that it represents the inability to handle it as a typed, auditable object rather than a silent gap or a runtime error.

L4L [6]: the work closest in motivation to LegalLean. The primary technical difference is that L4L's verification layer relies on SMT, while LegalLean's relies on Lean 4's kernel verifier. SMT provides more automation; Lean 4's kernel provides more auditability. L4L does not include a typed treatment of open texture.

LegalRuleML [12]: extends RuleML with deontic operators, temporal metadata, and defeasibility annotations. It is the most expressive knowledge representation format for legal rules but provides no automated reasoning. LegalLean and LegalRuleML address different problems: LegalLean verifies reasoning; LegalRuleML represents knowledge for interchange.

Hohfeldian frameworks: Prior work by McCarty [9] and Boella et al. [1] applies Hohfeld's fundamental legal relations in formal settings. LegalLean's three-modality deontic type is intentionally minimal; a Hohfeldian extension is straightforward but not required for the case studies.

Logic programming approaches: Kowalski and Sergot's Event Calculus [15], together with Sergot et al.'s British Nationality Act encoding [14], established the research programme of encoding law in logic programming. That tradition addressed defeasibility before DDL but did not handle open texture or produce machine-checked proofs.

7 Conclusion and Future Work

LegalLean demonstrates that Lean 4 dependent types provide a viable verification layer for legal reasoning systems. The key result is not the theorem count but the architecture: a system in which Hart's open texture is a first-class typed object rather than an external limitation, and in which the distinction between "machine-verified" and "requires human determination" is enforced by the type checker rather than stated in documentation.

The `monotonicity_break_general` theorem illustrates what machine-checked legal reasoning makes possible: a formal, auditable certificate that the TCJA 2017 removed a deduction permission for all home equity interest payments, not merely for specific test cases. This kind of structural certificate – not an answer, but a proof about the legal system itself – is what distinguishes a verifier from a transducer.

Several directions remain open. First, the transducer layer: language models are the natural tool for converting statutory text to Lean 4 stubs, with the type checker providing feedback to guide the formalisation. Second, zero-knowledge proofs for compliance: a party wishing to attest compliance without disclosing underlying facts could, in principle, use ZK proofs over the formal structure. Third, temporal algebra: a proper treatment of statutory effective dates, sunset provisions, and retroactivity requires a temporal type extending the current `TaxYear` structure. Fourth, scaling studies: encoding a complete statute rather than a selected kernel would test whether the proof engineering effort is tractable in practice.

The verification protocol that emerges from this work has three layers: the type checker verifies structural properties; the `FormalisationBoundary` annotations identify where institutional determination is required; and legal practitioners review the open-texture instances to ensure the core/penumbra analysis is sound. Only the first layer is automated. The claim is not that legal reasoning can be automated but that the parts that can be verified should be, and the parts that cannot should be typed rather than silently omitted.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Boella, G., van der Torre, L., Verhagen, H.: Introduction to normative multiagent systems. Computational and Mathematical Organization Theory **12**(2–3), 71–79 (2006)
2. Curry, H.B.: Functionality in combinatory logic. Proceedings of the National Academy of Sciences **20**(11), 584–590 (1934)
3. Governatori, G.: Representing business contracts in RuleML. International Journal of Cooperative Information Systems **14**(2–3), 181–216 (2005)
4. Governatori, G., Rotolo, A., Sartor, G.: Compliance in eHealth: a defeasible logic approach. Artificial Intelligence in Medicine **93**, 34–48 (2019)
5. Hart, H.L.: The Concept of Law. Oxford University Press (1961)
6. Hou, Z., Governatori, G., Hashmi, M., Lam, H.P.: Logic for law (L4L): a dependent-type-based framework for modelling and reasoning about law. In: Proceedings of the 34th International Conference on Legal Knowledge and Information Systems (JURIX 2021). Frontiers in Artificial Intelligence and Applications, IOS Press (2021)
7. Howard, W.A.: The formulae-as-types notion of construction. In: Seldin, J.P., Hindley, J.R. (eds.) To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism, pp. 479–490. Academic Press (1980)
8. Lawsky, S.B.: A logic for statutes. Florida Tax Review **21**(1), 60–113 (2017)
9. McCarty, L.T.: Reflections on TAXMAN: an experiment in artificial intelligence and legal reasoning. Harvard Law Review **90**(5), 837–893 (1977)
10. Mérigoux, D., Chataing, N., Protzenko, J.: Catala: a programming language for the law. In: Proceedings of the ACM on Programming Languages (ICFP 2021). ACM (2021)

11. de Moura, L., Selsam, D., Ullrich, S., Avigad, J.: The Lean 4 theorem prover and programming language. In: Proceedings of the 28th International Conference on Automated Deduction (CADE-28). Lecture Notes in Computer Science, Springer (2021)
12. Palmirani, M., Governatori, G.: Modelling legal knowledge for GDPR compliance checking. In: Proceedings of the 31st International Conference on Legal Knowledge and Information Systems (JURIX 2018). Frontiers in Artificial Intelligence and Applications, IOS Press (2018)
13. Reiter, R.: A logic for default reasoning. Artificial Intelligence **13**(1–2), 81–132 (1980)
14. Sergot, M.J., Sadri, F., Kowalski, R.A., Kriwaczek, F., Hammond, P., Cory, H.T.: The British Nationality Act as a logic program. Communications of the ACM **29**(5), 370–386 (1986)
15. Shanahan, M.: The event calculus explained. In: Artificial Intelligence Today, Lecture Notes in Computer Science, vol. 1600, pp. 409–430. Springer (1999)